RELATÓRIO · TRABALHO PRÁTICO 1 · ISI

Análise e Visualização de Dados Telemétricos de Fórmula 1

ETL (Extract, Transform, Load) com Pentaho Data Integration (Kettle)

Dados obtidos: Grande Prémio de Singapura 2023

Autor

Nuno da Cunha Faria Gajo (a23002)

Unidade Curricular

Integração de Sistemas de Informação

Curso

Licenciatura em Engenharia de Sistemas Informáticos

1. Enquadramento

Este relatório documenta o trabalho prático desenvolvido no âmbito da unidade curricular de ISI, cujo tema escolhido é a **análise e visualização de dados telemétricos de Fórmula 1**. O trabalho consiste na construção de um processo de integração de dados (ETL) com o Pentaho Data Integration (Kettle), aplicado a dados reais do Grande Prémio de Singapura (F1) de 2023.

Campo	Informação
Equipa	*Nuno da Cunha Faria Gajo*
Unidade Curricular	*ISI – Integração de Sistemas de Informação*
Curso	*Licenciatura em Engenharia de Sistemas Informáticos*
Instituição	*Universidade Politécnica do Cávado e do Ave (UPCA)*
Docente	*Óscar Ribeiro*
Ano letivo	2026/2027

2. Problema

O objetivo deste trabalho é **demonstrar um processo completo de tratamento de dados telemétricos**: obter dados de fontes remotas (ficheiros e API), detetar e remover erros, transformar os dados num formato comum, e comunicar automaticamente o resultado da filtragem. Numa situação real, os dados originais raramente estão prontos a ser analisados: podem conter linhas repetidas, valores impossíveis, campos mal formatados ou estar fora de ordem, e qualquer visualização construída sobre eles seria enganadora. Tendo também em conta o tema do trabalho, dados telemétricos e em geral dados de sensores automóveis e de sistemas embebidos são bastante extensos, e tendem a obter dados decorridos das leituras analógicas e eletrónicas.

2.1 Dados de origem

Os dados foram obtidos a partir da API pública OpenF1, que disponibiliza dados de telemetria, meteorologia e outras informações de corrida:

- **Telemetria** (piloto 55 - Carlos Sainz Jr., sessão 9159 – primeira sessão de treino do dia 15 de Setembro de 2023): `https://api.openf1.org/v1/car_data?driver_number=55&session_key=9159`
- **Meteorologia** (Grande Prémio de Singapura, Circuito Urbano de Marina Bay): `https://api.openf1.org/v1/weather?meeting_key=1219`
- **Documentação**: `https://openf1.org`

O ponto de partida foram dois ficheiros:

Ficheiro	Conteúdo
`car_data.csv`	Telemetria do piloto 55 com 18 470 registos. Campos: brake, date, driver_number, drs, meeting_key, n_gear, rpm, session_key, speed e throttle.

Ficheiro	Conteúdo
`singapore_gp_weather.json`	Dados meteorológicos: temperatura do ar e da pista, humidade, pressão, velocidade e direção do vento e chuva.

2.2 Introdução de erros

Os ficheiros originais não tinham erros, pelo que **foram introduzidos erros de forma artificial e controlada** (com um script) em cópias dos ficheiros, designadas com o sufixo `_sujo`. Assim, é possível ter proveito das transformações do Kettle para detetar e corrigir erros de dados. No ficheiro de telemetria foram introduzidos os seguintes erros:

Tipo de erro	Quantidade
Velocidade (`speed`) com texto (por exemplo, "abc" ou "N/A")	60
Velocidade negativa	40
Velocidade fora do intervalo plausível	30
Datas mal formatadas ("ontem", vazias, 15/09/2023, mês 13)	50
Rotações (`rpm`) em branco	40
Linhas duplicadas	150
Ordem dos registos alterada aleatoriamente	todos
Total de linhas do ficheiro sujo (18 470 originais + 150 duplicadas)	**18 620**

No ficheiro meteorológico, como o original só tinha 3 registos, foram **acrescentados 21 registos simulados** (pequena variação dos valores reais) para obter um maior número de registos. Foram também introduzidos 5 registos com erros (texto na temperatura do ar, humidade de 150 %, data mal formatada, temperatura da pista nula e vento negativo) e 3 registos duplicados, num total de 27 registos.

2.3 Objetivos específicos

- Ler dados remotos nos formatos CSV e JSON, e obter dados diretamente de uma API.
- Detetar e remover dados duplicados e mal formatados, filtrar valores impossíveis e ordenar os registos.
- Converter os dados para um formato comum (CSV), pronto para visualização.
- Contabilizar o resultado da limpeza e enviá-lo por email, em percentagem, através de um job.
- Visualizar os dados limpos num dashboard em Node-RED, com gráficos da telemetria (velocidade, rotações, acelerador e mudança) e da velocidade máxima.

3. Estratégia utilizada

3.1 Visão geral do processo

Cada ficheiro é tratado por uma transformação própria, com a mesma lógica: ler, validar, filtrar, remover duplicados, ordenar e exportar. Os registos rejeitados em cada fase não são simplesmente descartados, mas sim encaminhados para um passo dedicado, o que permite contá-los e produzir o resumo final. O processo segue as seguintes etapas:

1. **Leitura** do ficheiro com todos os campos como texto, para que os erros de formato não façam falhar o Kettle.
2. **Conversão de tipos** dos campos numéricos, o que não converte é rejeitado.
3. **Filtragem** de valores impossíveis ou em falta (velocidades negativas ou acima de 400 km/h, rotações vazias).
4. **Validação das datas** com uma expressão regular.
5. **Remoção de duplicados**, comparando a linha inteira.
6. **Ordenação** cronológica pelo campo `date`.
7. **Exportação** dos dados limpos para um ficheiro CSV.
8. **Contagem e resumo**: cálculo do número e da percentagem de linhas em cada categoria.
9. **Envio por email** do resumo, orquestrado por um job.
10. **Visualização** dos dados limpos num dashboard em Node-RED (Dashboard 2.0).

3.2 Operadores utilizados

Passo do Kettle	Categoria	Função neste trabalho
CSV file input	Input	Lê o ficheiro de telemetria.
JSON input	Input	Lê o ficheiro meteorológico e a resposta da API, com os campos definidos à mão.
Generate rows	Input	Gera uma linha para despoletar a chamada à API.
REST client	Lookup	Faz o pedido HTTP GET à API OpenF1.
Select values	Transform	Converte tipos: o tratamento de erros encaminha para os rejeitados o que não for convertido.
Filter rows	Flow	Aplica condições (intervalos, valores nulos) e a expressão regular das datas.
Unique rows (HashSet)	Statistics	Deteta linhas repetidas e redireciona-as.
Sort rows	Transform	Ordena por data e, no resumo, por categoria.
Text file output	Output	Exporta os dados limpos para um ficheiro CSV.
Dummy	Flow	Recolhe os registos rejeitados em cada fase.
Group by	Statistics	Conta as linhas de cada destino.
Add constants	Transform	Etiqueta cada contagem com uma categoria.

Passo do Kettle	Categoria	Função neste trabalho
Modified JavaScript value	Scripting	Calcula o total e as percentagens.
Set variables	Job	Guarda os resultados em variáveis, usadas no email.
Start, Transformation, Mail, Success	Job (entradas)	Orquestram a execução e o envio do email.

Para a parte de visualização foram ainda utilizados os seguintes nós do Node-RED, com a extensão Dashboard 2.0:

Nó do Node-RED	Categoria	Função neste trabalho
inject	common	Dispara o fluxo ao arrancar ou manualmente.
file in	storage	Lê o ficheiro CSV limpo gerado pelo Kettle.
csv	parser	Converte o texto do CSV em objetos, um por registo.
debug	common	Mostra o conteúdo das mensagens, para verificar a leitura.
function	function	Converte as datas, reduz os pontos e prepara os dados dos gráficos.
ui-chart	Dashboard 2.0	Desenha os gráficos de linha.
ui-gauge	Dashboard 2.0	Mostra a velocidade máxima num indicador.

3.3 Decisões de conceção

- **Tudo como texto na leitura.** Se os campos numéricos fossem lidos como inteiros, o Kettle falharia nas linhas com texto. Lendo como texto, a conversão é feita num passo à parte, onde os erros são tratados.
- **Rejeitar em vez de apagar.** Cada fase envia as linhas inválidas para um passo próprio, o que permite saber exatamente quantas foram removidas e porquê.
- **Ordem das operações.** As validações vêm antes da remoção de duplicados, para que estes sejam apurados apenas sobre registos válidos, e a ordenação é feita no fim.
- **Datas como texto.** O formato ordena corretamente como texto, e o ficheiro tem datas com e sem milissegundos, o que impede o uso de um único formato de conversão.
- **Resumo através de variáveis e de um job.** As variáveis calculadas na transformação só ficam disponíveis para o email quando a transformação corre dentro de um job.
- **Gráficos com menos pontos.** O dashboard usa um registo em cada 20 (cerca de 900 pontos por gráfico), porque desenhar os 18 250 registos tornaria os gráficos lentos sem acrescentar informação visível.

4. Transformações

4.1 Limpeza do ficheiro de telemetria (CSV)

A transformação principal lê o ficheiro `car_data_sujo.csv` e aplica, por esta ordem: a conversão de tipos, dois filtros, a remoção de duplicados e a ordenação, gravando o resultado em `car_data_limpo.csv`. Os três passos de baixo recolhem os registos rejeitados em cada fase.

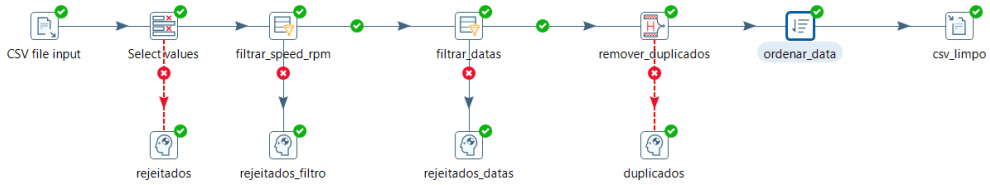


Figura 1 – *Diagrama completo da transformação de limpeza do CSV, de CSV file input até csv_limpo.*

4.1.1 Leitura do ficheiro (CSV file input)

O primeiro passo lê o ficheiro `car_data_sujo.csv`, com a vírgula como delimitador e com a primeira linha como cabeçalho. Os 10 campos (brake, date, driver_number, drs, meeting_key, n_gear, rpm, session_key, speed e throttle) foram todos definidos como **String**. Isto é propositado: assim o Kettle não falha nas linhas com texto em campos que deviam ser numéricos, e a validação fica a cargo dos passos seguintes.

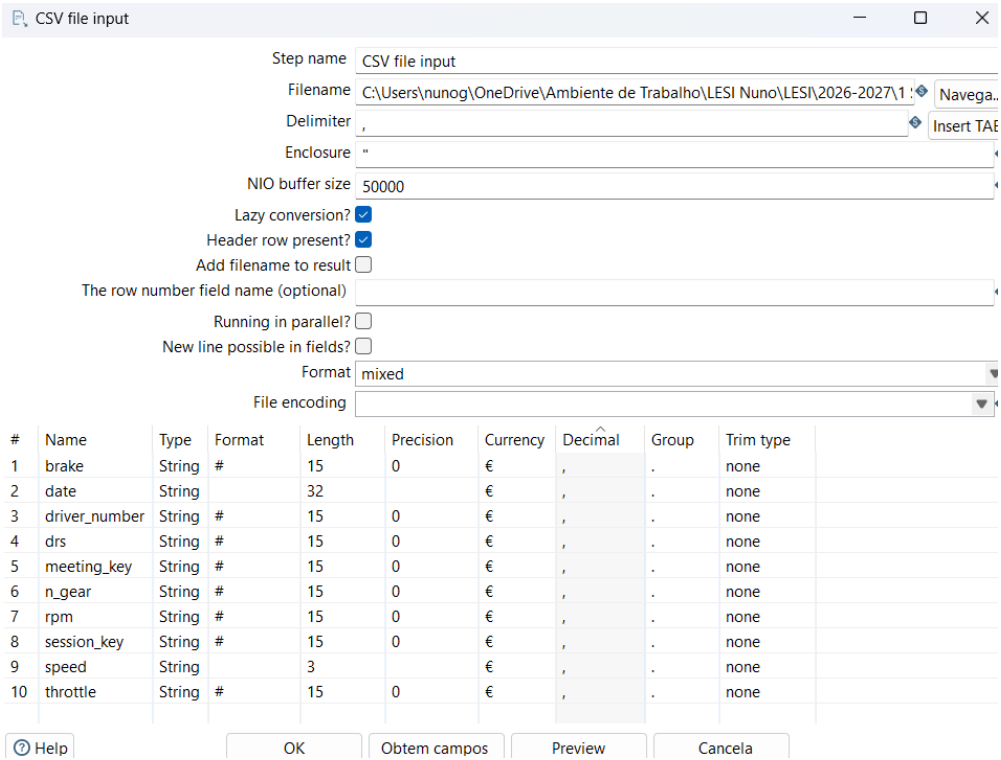


Figura 2 – *Configuração do passo CSV file input, com todos os campos lidos como String.*

4.1.2 Conversão de tipos (Select values)

O passo *Select values* converte os campos `speed` e `rpm` para inteiros (separador *Meta-data*). Os registos

em que a conversão falha, por terem texto no campo, são enviados pelo tratamento de erros (*error handling*) para o passo `rejeitados`. O campo `date` não é convertido aqui, porque é validado separadamente.

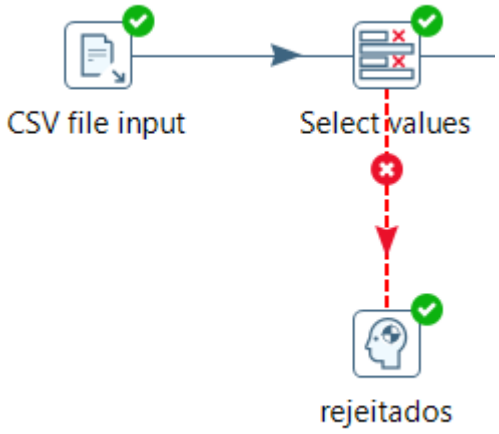


Figura 3 – *CSV file input ligado ao Select values; a seta vermelha tracejada leva os erros ao passo rejeitados.*

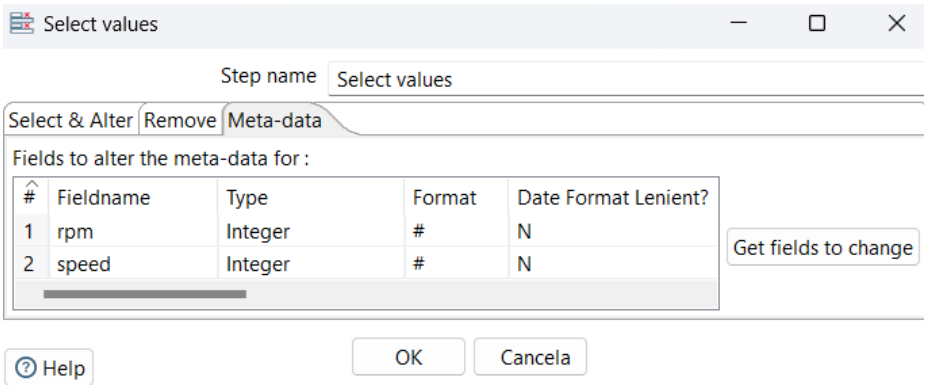


Figura 4 – *Meta-data do Select values: rpm e speed convertidos para Integer.*

4.1.3 Filtragem de velocidades e rotações inválidas (Filter rows)

O passo `filtrar_speed_rpm` aplica a condição `(speed >= 0 AND speed <= 400) AND rpm IS NOT NULL`. Remove as velocidades negativas e os valores extremos (impossíveis para um carro de Fórmula 1), bem como os registos sem rotações. As linhas que cumprem a condição seguem para `filtrar_datas`; as restantes vão para `rejeitados_filtro`.

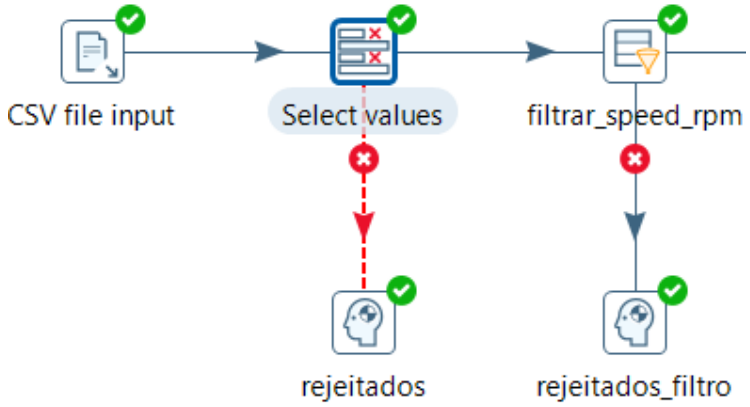


Figura 5 – *Diagrama com o passo filtrar_speed_rpm e o destino rejeitados_filtro.*

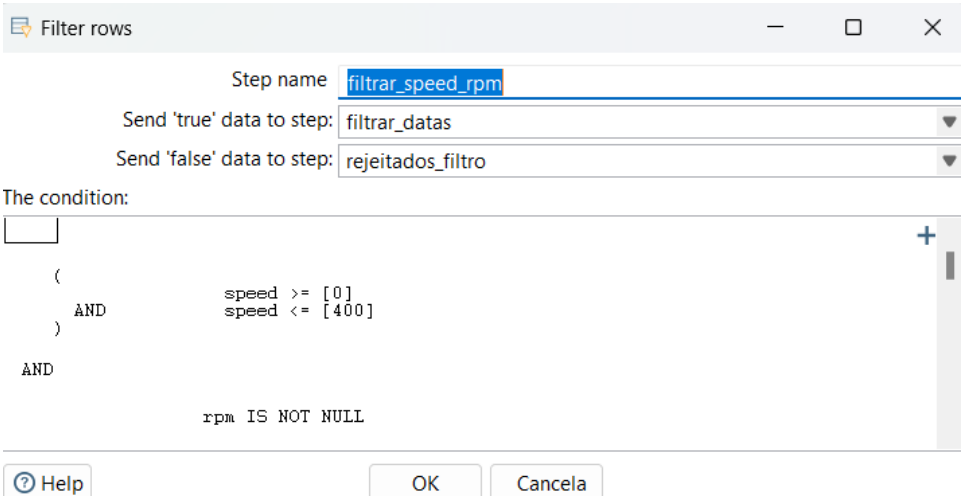


Figura 6 – *Condição do passo filtrar_speed_rpm.*

4.1.4 Validação das datas (Filter rows com expressão regular)

O passo `filtrar_datas` valida o formato do campo `date` com a função REGEXP e a expressão `^2023-09-[0-9]{2} ([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]([.][0-9]{1,6})?[+]00:00$`. A expressão aceita datas no formato ISO, com ou sem milissegundos, em setembro de 2023, com horas entre 00 e 23 e minutos e segundos entre 00 e 59. Rejeita valores como "ontem", campos vazios, datas noutro formato (por exemplo, 15/09/2023) e datas impossíveis (por exemplo, mês 13). Os registos inválidos seguem para `rejeitados_datas`.

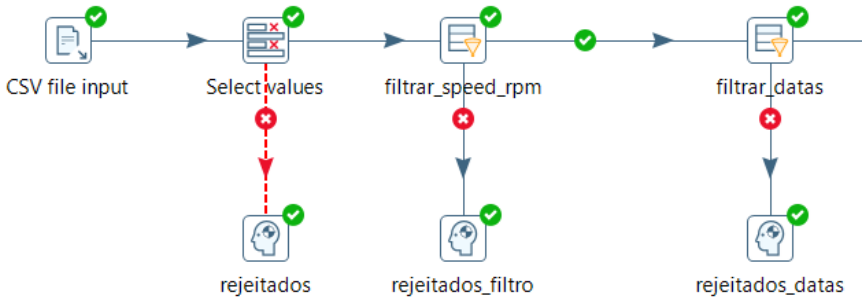


Figura 7 – *Diagrama com o passo filtrar_datas e o destino rejeitados_datas.*

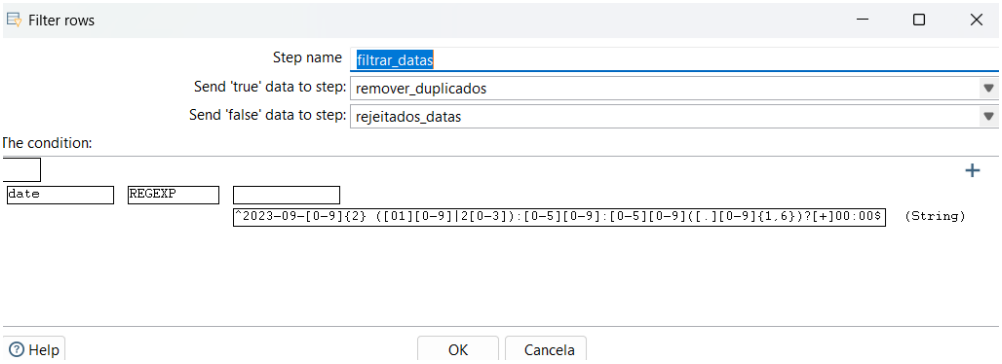


Figura 8 – *Condição do passo filtrar_datas: campo date, função REGEXP e a expressão regular.*

4.1.5 Remoção de duplicados (Unique rows)

O passo *Unique rows (HashSet)*, chamado `remover_duplicados`, compara **todos os campos** de cada linha (a tabela de campos fica vazia, o que significa comparar a linha completa) e mantém apenas a primeira ocorrência. Com a opção *Redirect duplicate row* ativa, as linhas repetidas são redirecionadas pelo tratamento de erros para o passo `duplicados`, o que permite contá-las. Esta operação é feita depois das validações, para que os duplicados sejam apurados apenas sobre registos válidos.

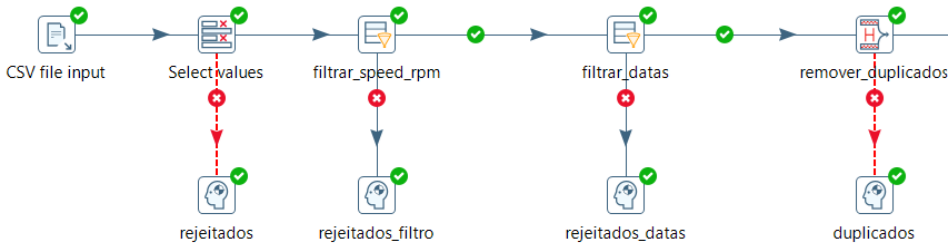


Figura 9 – *Diagrama com o passo remover_duplicados e o destino duplicados.*

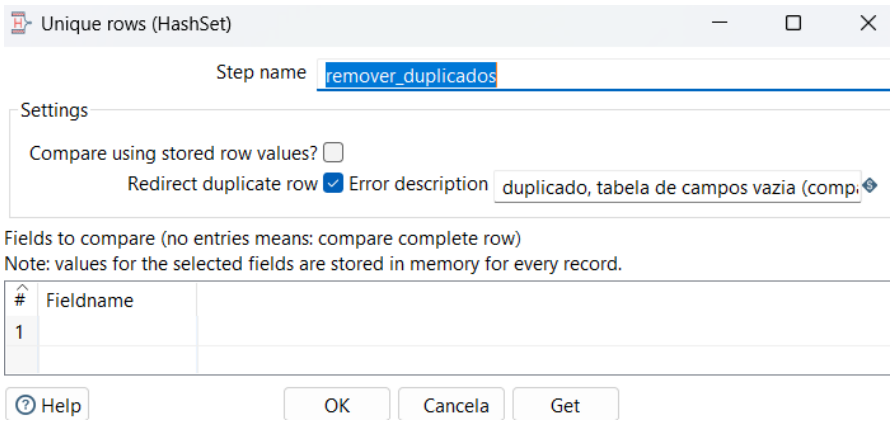


Figura 10 – *Configuração do Unique rows (HashSet) com Redirect duplicate row ativo.*

4.1.6 Ordenação (Sort rows)

O passo `ordenar_data` ordena os registos pelo campo `date`, de forma ascendente. Como as datas estão em formato ISO, a ordenação como texto coincide com a ordem cronológica, o que repõe a ordem original que foi baralhada ao introduzir os erros.

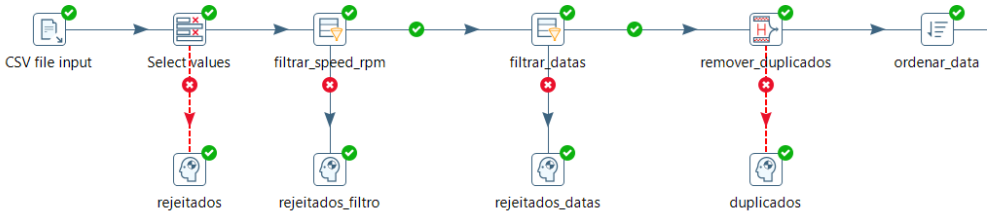


Figura 11 – *Diagrama até ao passo ordenar_data.*

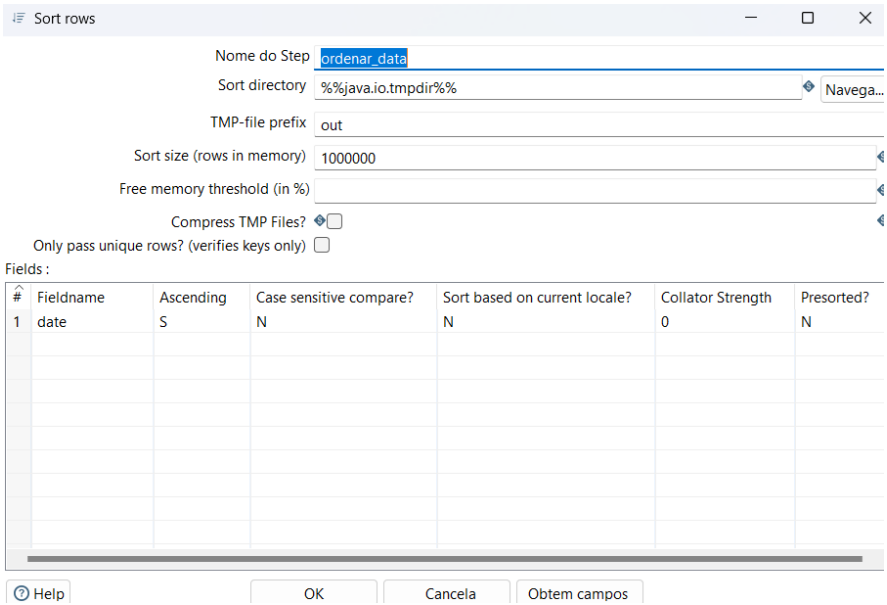


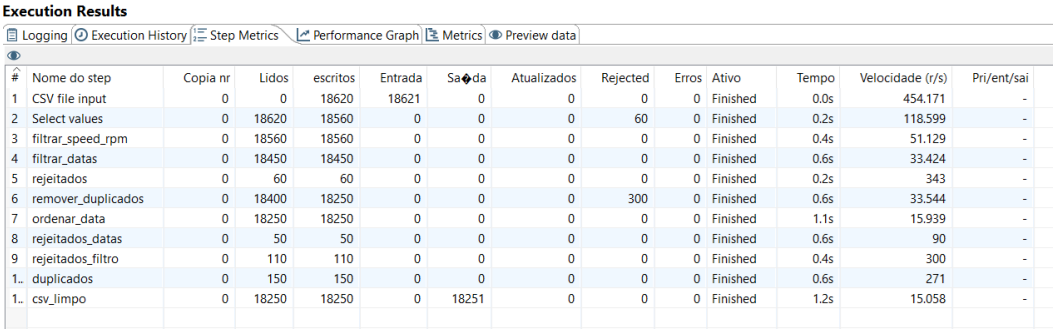
Figura 12 – *Configuração do Sort rows: campo date, ordem ascendente.*

4.1.7 Exportação (Text file output)

Por fim, os registos limpos, filtrados e ordenados são exportados, através do passo `csv_limpo` (*Text file output*), no ficheiro `car_data_limpo.csv`, em formato CSV com cabeçalho.

4.1.8 Resultados

O separador *Step Metrics* do Kettle mostra quantas linhas passaram por cada passo. Os valores obtidos coincidem com os erros que foram introduzidos.



Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa◆da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	CSV file input	0	0	18620	18621	0	0	0	0	Finished	0.0s	454.171	-
2	Select values	0	18620	18560	0	0	0	60	0	Finished	0.2s	118.599	-
3	filtrar_speed_rpm	0	18560	18560	0	0	0	0	0	Finished	0.4s	51.129	-
4	filtrar_datas	0	18450	18450	0	0	0	0	0	Finished	0.6s	33.424	-
5	rejeitados	0	60	60	0	0	0	0	0	Finished	0.2s	343	-
6	remover_duplicados	0	18400	18250	0	0	0	300	0	Finished	0.6s	33.544	-
7	ordenar_data	0	18250	18250	0	0	0	0	0	Finished	1.1s	15.939	-
8	rejeitados_datas	0	50	50	0	0	0	0	0	Finished	0.6s	90	-
9	rejeitados_filtro	0	110	110	0	0	0	0	0	Finished	0.4s	300	-
1..	duplicados	0	150	150	0	0	0	0	0	Finished	0.6s	271	-
1..	csv_limpo	0	18250	18250	0	18251	0	0	0	Finished	1.2s	15.058	-

Figura 13 – *Step Metrics da execução: linhas lidas e escritas por cada passo.*

Etapa	Linhas à entrada	Removidas	Linhas à saída
Leitura do ficheiro	18 620	–	18 620
Select values (texto em `speed`)	18 620	60	18 560
filtrar_speed_rpm (intervalo e `rpm` vazio)	18 560	110	18 450
filtrar_datas (datas inválidas)	18 450	50	18 400
remover_duplicados	18 400	150	18 250
Total	**18 620**	**370**	**18 250**

Categoria	Linhas	% do total
Dados corretos (ficheiro limpo)	18 250	98,0 %
Texto em `speed`	60	0,3 %
Velocidade fora do intervalo ou `rpm` vazio	110	0,6 %
Datas inválidas	50	0,3 %
Duplicados	150	0,8 %

Conclusão

O Kettle detetou todos os erros introduzidos: 60 + 110 + 50 + 150 = 370 linhas removidas (2,0 %), restando 18 250 registos corretos (98,0 %). O número de linhas rejeitadas em cada passo coincide exatamente com o número de erros de cada tipo.

4.2 Limpeza do ficheiro meteorológico (JSON)

O ficheiro meteorológico é tratado com uma transformação equivalente à anterior, com algumas diferenças, impostas pelo formato e pelos campos:

- **Leitura (JSON input):** ao contrário do CSV, os 8 campos (date, air_temperature, track_temperature, humidity, pressure, wind_speed, wind_direction e rainfall) têm de ser

definidos manualmente, com os caminhos `$[*].campo`, e foram lidos como String.

- **Conversão (Select values):** os campos decimais (temperaturas, humidade, pressão e velocidade do vento) são convertidos para *Number*, com o formato `#.#`, o ponto como separador decimal e a vírgula como separador de milhares. `wind_direction` é convertido para *Integer*. Indicar o ponto como separador decimal é essencial, porque o Windows em português usa a vírgula e o Kettle leria mal valores como 31.5.
- **Filtro (Filter rows):** as condições são específicas dos dados: humidade entre 0 e 100, velocidade do vento maior ou igual a zero e temperatura da pista não nula.
- **Saída:** em vez de voltar a gravar JSON, os dados limpos são gravados em **CSV**, o que constitui uma transformação de JSON para CSV.

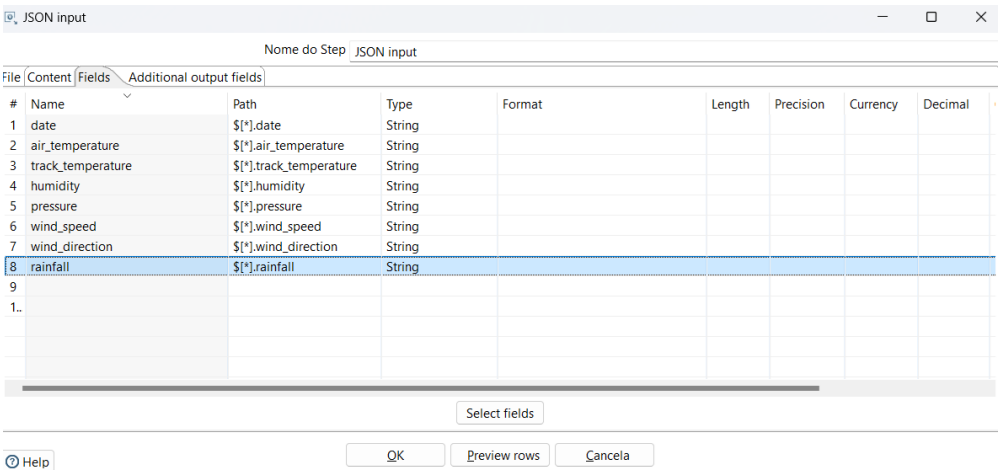


Figura 14 – *Campos do JSON input, definidos manualmente com os caminhos $[*].campo.*

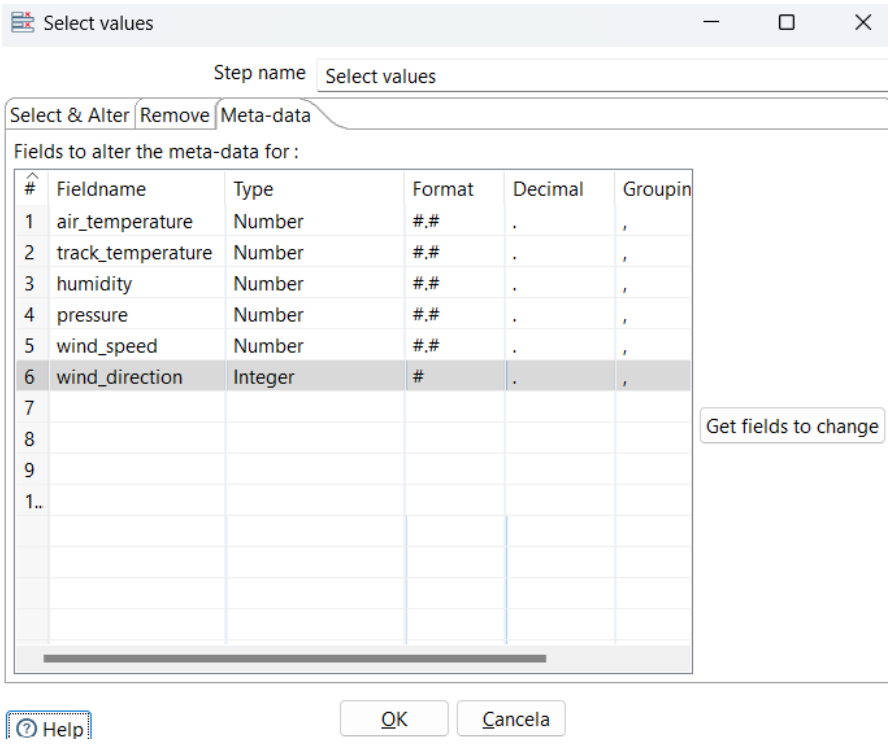


Figura 15 – *Select values do ficheiro meteorológico: campos decimais em Number e wind_direction em Integer.*

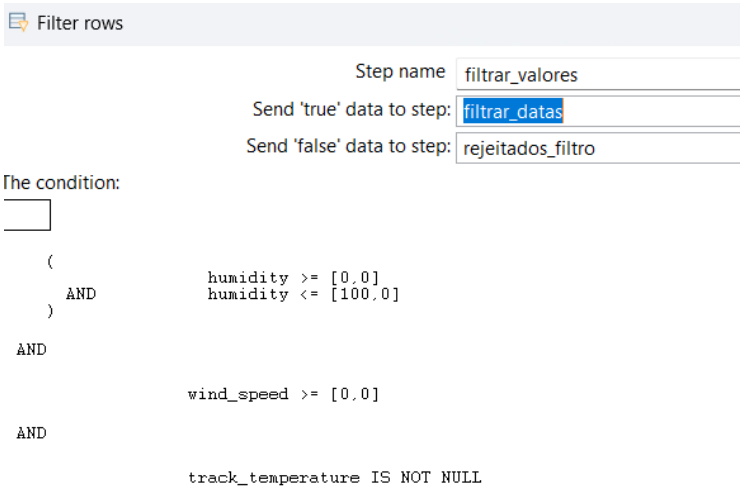


Figura 16 – *Condições do filtro filtrar_valores: humidade, velocidade do vento e temperatura da pista.*

O resto da transformação (validação das datas, remoção de duplicados, ordenação e gravação) é igual à do CSV. O resultado é o seguinte:

Categoria	Registos	% do total
Dados corretos	19	70,4 %
Mal formatados ou fora do intervalo (texto, humidade, vento, nulo, data)	5	18,5 %
Duplicados	3	11,1 %
Total lido	**27**	**100 %**

4.3 Obtenção de dados através de uma API

Até aqui os dados foram lidos através de ficheiros remotos, guardados localmente manualmente. Para explorar a funcionalidade de obter dados remotos, foi construída uma transformação que importa dados diretamente da API OpenF1. Escolheu-se a telemetria da **mesma sessão (9159) mas do piloto número 44**. Como estes dados vêm da API já limpos, não foi necessário aplicar validações: a transformação limita-se a obter os dados, lê-los e gravá-los em CSV.



Figura 17 – *Diagrama da transformação: Generate rows, REST client, JSON input e Text file output.*

O passo **Generate rows** existe apenas para inicializar o fluxo, gerando a linha que dispara o pedido.

No passo **REST client** indica-se o URL da API (`https://api.openf1.org/v1/car_data?driver_number=44&session_key=9159`), o método GET, e definem-se os nomes dos campos de saída: `result` para o conteúdo da resposta e `codigo` para o código de estado HTTP.

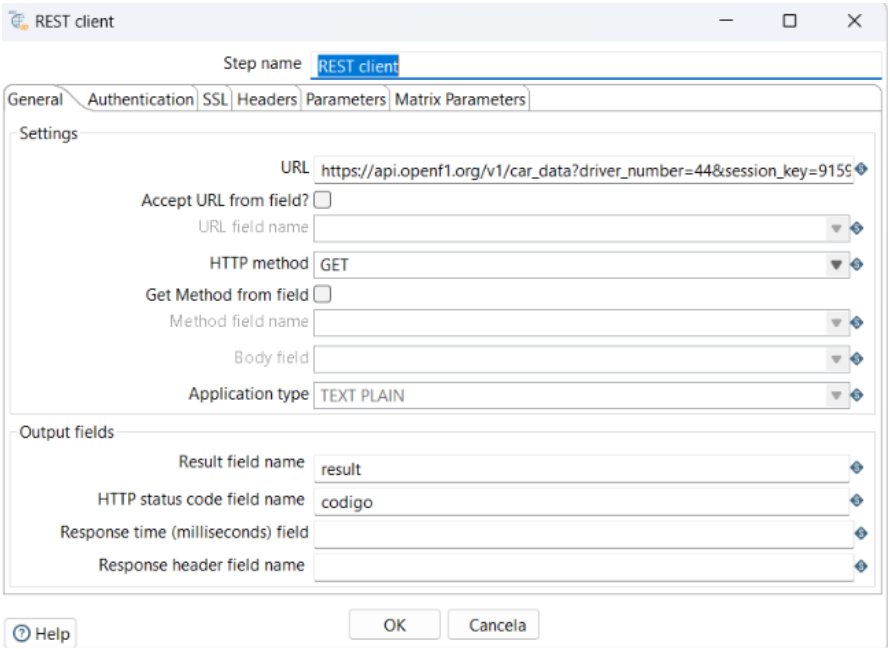


Figura 18 – *Configuração do REST client: URL com driver_number=44 e session_key=9159.*

No **JSON input** indica-se que a origem dos dados é um campo do passo anterior (`result`), em vez de um ficheiro, e acrescentam-se manualmente os 10 campos.

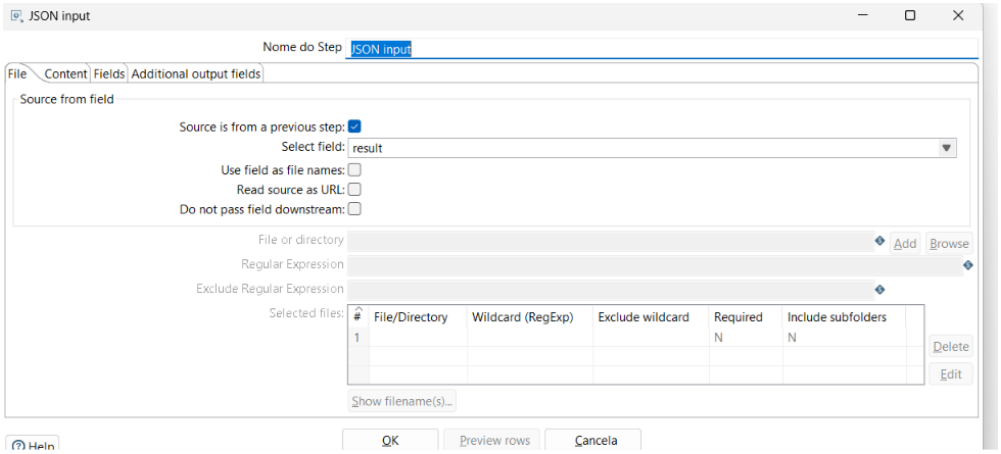


Figura 19 – *JSON input com a origem definida a partir de um campo do passo anterior (result).*

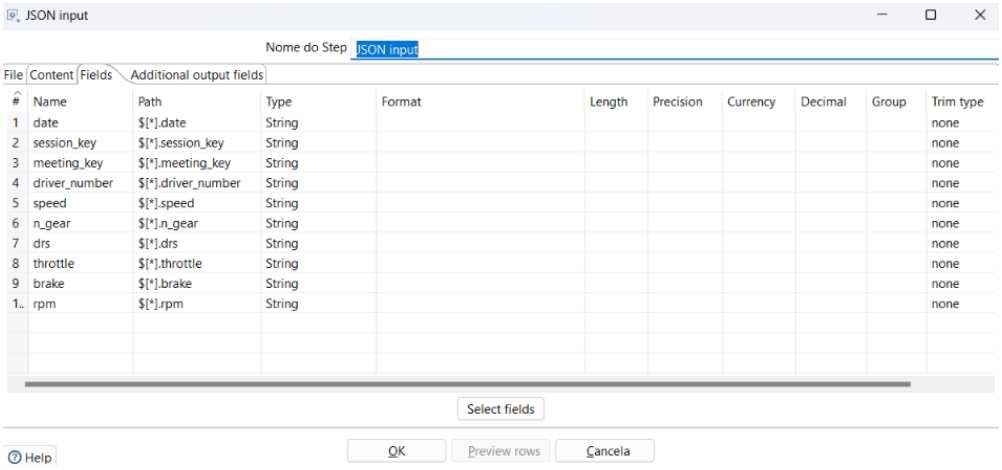


Figura 20 – *Campos lidos da resposta da API: date, session_key, meeting_key, driver_number, speed, n_gear, drs, throttle, brake e rpm.*

Por fim, os dados são gravados num ficheiro CSV com o passo *Text file output*, tal como nas transformações anteriores.

4.4 Cálculo do resumo da limpeza

Para enviar por email a percentagem de dados corretos, mal formatados e duplicados, a transformação de limpeza foi estendida com um bloco que conta as linhas de cada destino e calcula as percentagens. O bloco funciona do seguinte modo: cada um dos cinco destinos (ficheiro limpo, rejeitados, rejeitados_filtro, rejeitados_datas e duplicados) alimenta uma contagem, as cinco contagens são reunidas num só fluxo, e um script calcula os resultados, que ficam guardados em variáveis.

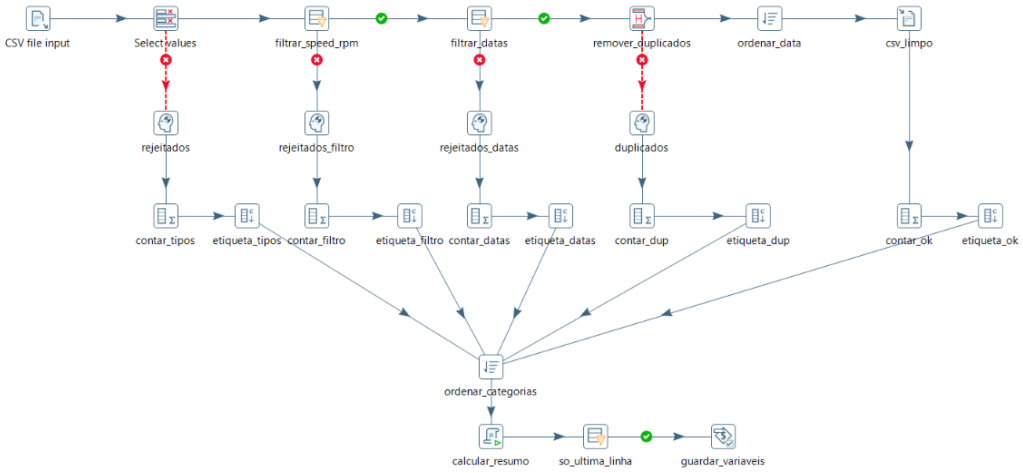


Figura 21 – *Transformação completa, com o bloco de resumo na parte inferior.*

Contagem e etiquetagem

Cada destino liga a um passo **Group by** sem campos de agrupamento, que conta as linhas e devolve o resultado no campo `n`. A opção *Always give back a result row* garante que, mesmo que um destino não receba nenhuma linha, a contagem devolve `n = 0` em vez de não devolver nada. A seguir, um passo **Add constants** acrescenta um campo `categoria` com o valor que identifica o destino.

Destino	Passos de contagem	Valor de categoria
Ficheiro limpo	contar_ok → etiqueta_ok	1_OK
rejeitados (texto em campo numérico)	contar_tipos → etiqueta_tipos	2_TIPOS
rejeitados_filtro	contar_filtro → etiqueta_filtro	3_FILTRO
rejeitados_datas	contar_datas → etiqueta_datas	4_DATAS
duplicados	contar_dup → etiqueta_dup	5_DUP

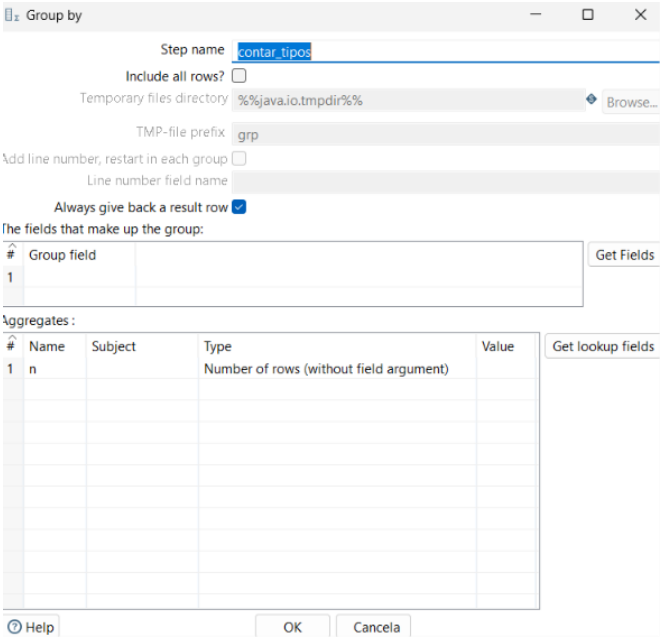


Figura 22 – *Group by contar_tipos: sem campos de agrupamento, agregado n e Always give back a result row ativo.*

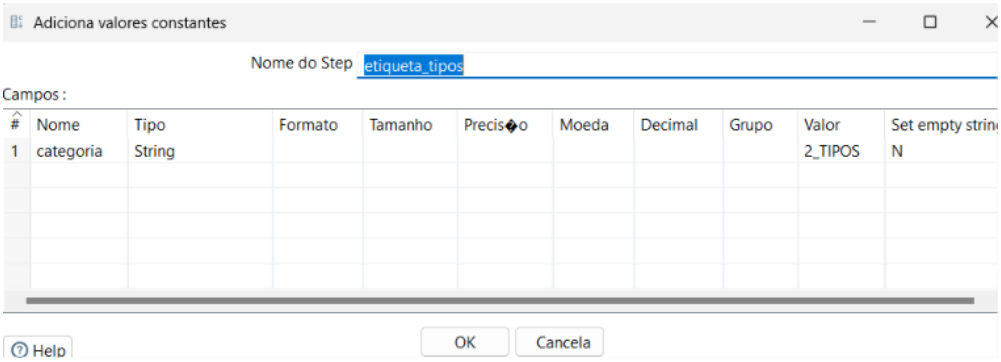


Figura 23 – *Add constants etiqueta_tipos: campo categoria do tipo String com o valor 2_TIPOS.*

Ordenação, cálculo e variáveis

As cinco contagens entram num passo **Sort rows** (`ordenar_categorias`) que as ordena por `categoria`. Isto garante que a categoria `5_DUP` chega em último, quando as outras quatro já foram lidas. O passo **Modified JavaScript value** (`calcular_resumo`) guarda cada contagem e, ao receber a última, calcula o total, o número de linhas mal formatadas (a soma das três categorias de rejeitados) e as três percentagens, devolvendo-os em campos.

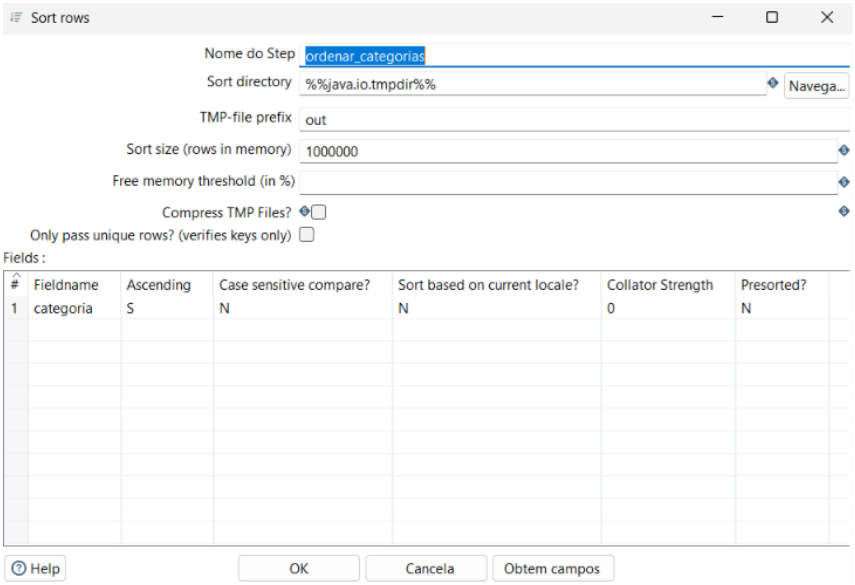


Figura 24 – *Sort rows ordenar_categorias, ordenado pelo campo categoria.*

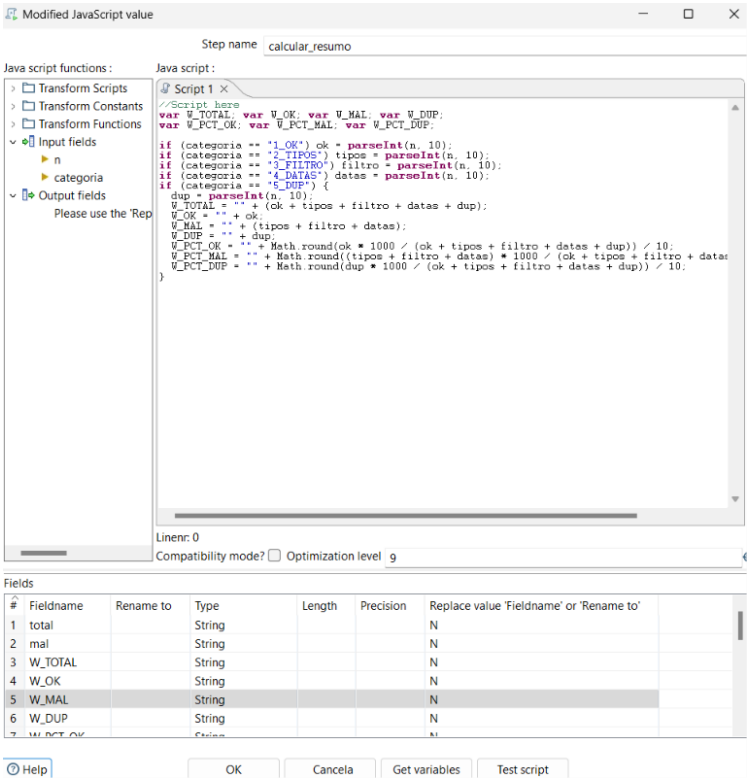


Figura 25 – *Script do passo calcular_resumo e respetivos campos de saída.*

Como só a linha `5_DUP` contém os valores calculados, o **Filter rows** `so_ultima_linha` deixa passar apenas essa linha (`categoria = 5_DUP`) para o **Set variables** `guardar_variaveis`, que guarda cada campo numa variável com âmbito *Valid in the root job*, ficando assim disponível para o job que envia o email.

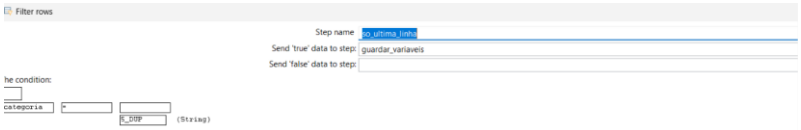


Figura 26 – *Filter rows so_ultima_linha: só a linha com categoria 5_DUP segue para o Set variables.*

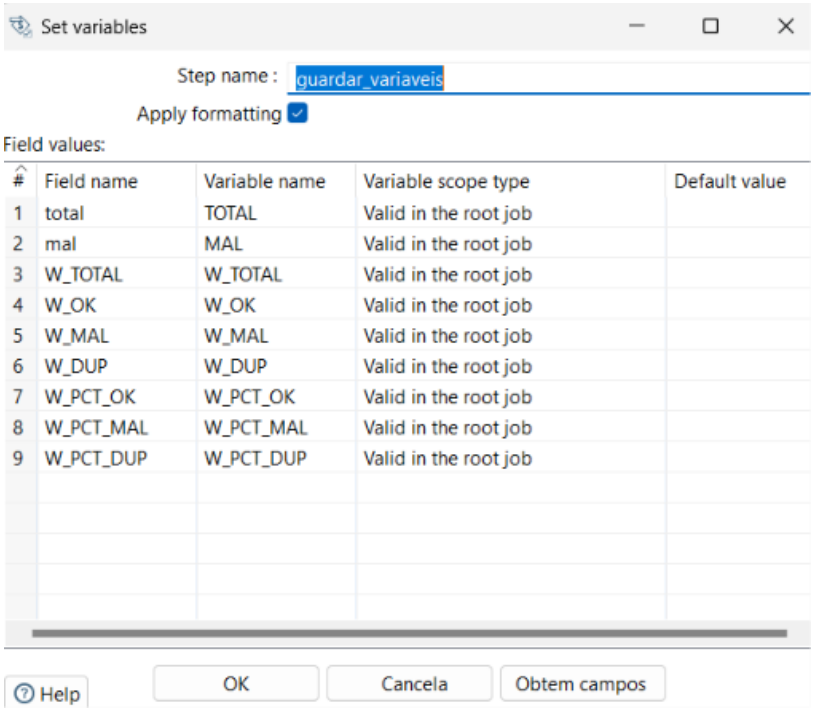


Figura 27 – *Set variables guardar_variaveis: cada campo é guardado numa variável com âmbito root job.*

4.5 Visualização dos dados: dashboard no Node-RED

Para visualizar os dados limpos foi construído um dashboard no **Node-RED**, com a extensão **Dashboard 2.0** (pacote `@flowfuse/node-red-dashboard`). O fluxo lê o ficheiro `car_data_limpo.csv`, produzido pela transformação do Kettle, e apresenta a evolução da velocidade, das rotações, do acelerador e da alteração das mudanças ao longo da sessão, além de um indicador com a velocidade máxima. Assim, o Kettle fica responsável por obter e limpar os dados, e o Node-RED por os mostrar.

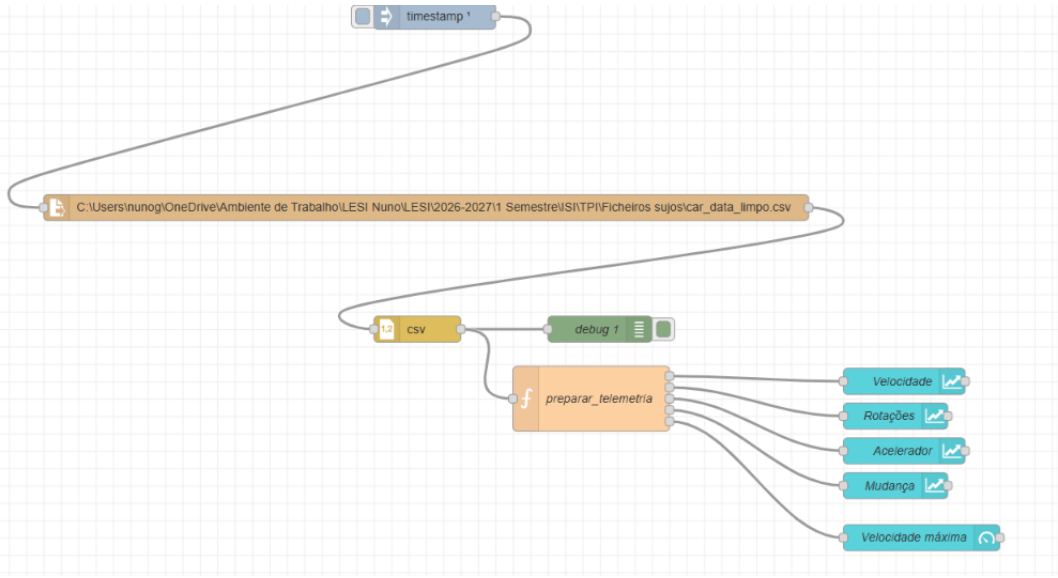


Figura 28 – *Fluxo do Node-RED: leitura do CSV limpo (inject, file in, csv), preparação dos dados (function), quatro gráficos e um indicador.*

4.5.1 Preparação do ambiente

O Node-RED corre sobre o Node.js e foi instalado globalmente com o comando `npm install -g --unsafe-perm node-red`. Depois de arrancar com o comando `node-red`, o editor fica disponível em

`http://127.0.0.1:1880`. O Dashboard 2.0 foi instalado a partir do próprio editor, em *Manage palette* > *Install*, pesquisando por `@flowfuse/node-red-dashboard`. Este pacote é o sucessor do Node-RED Dashboard original (`node-red-dashboard`), que está descontinuado, e por isso é o utilizado. O dashboard fica acessível em `http://127.0.0.1:1880/dashboard`.

4.5.2 Leitura do ficheiro CSV limpo

A leitura é feita por uma cadeia de três nodes. O node **inject** arranca o fluxo automaticamente 0.1 segundos depois de este ser publicado (*Deploy*). Também pode ser disparado manualmente, através do botão à esquerda do node.

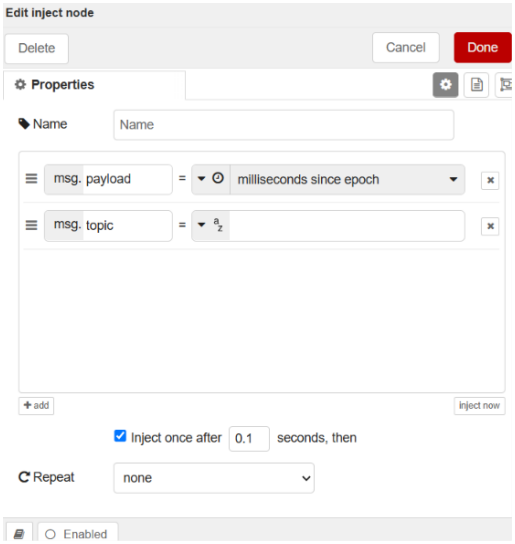


Figura 29 – *Node inject: arranca o fluxo 0.1 segundos depois do Deploy.*

O node **file in** lê o ficheiro `car_data_limpo.csv`, gravado pelo Kettle, a partir do seu caminho absoluto, e devolve o conteúdo como uma única string.

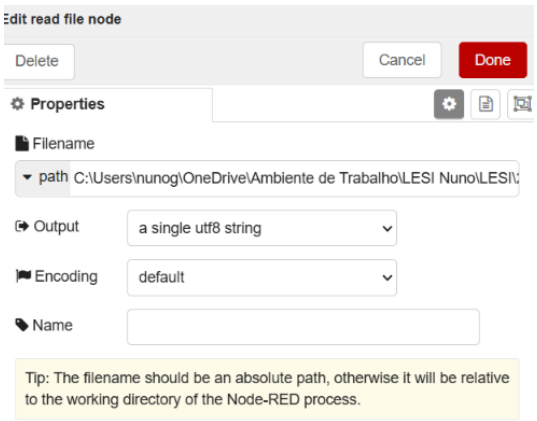


Figura 30 – *Node file in: caminho do ficheiro e saída como uma única string.*

O node **csv** converte esse texto em objetos. O separador é o **ponto e vírgula** (*semicolon*), porque é o separador que o passo *Text file output* do Kettle usa por defeito. A primeira linha contém os nomes das colunas, os valores numéricos são convertidos em números e o resultado sai numa única mensagem com um array. Neste caso, o array tem 18250 objetos, um por registo, com os campos `brake`, `date`,

`driver_number`, `drs`, `meeting_key`, `n_gear`, `rpm`, `session_key`, `speed` e `throttle`. Um node **debug** ligado a esta saída permitiu confirmar o conteúdo lido.

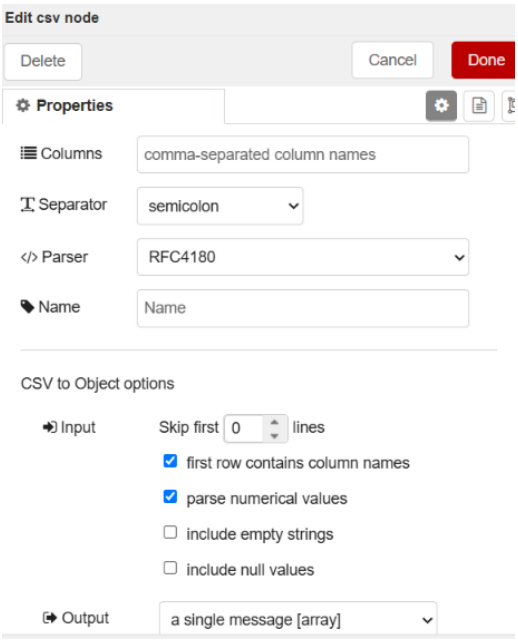


Figura 31 – *Node csv: separador ponto e vírgula, primeira linha com os nomes das colunas e saída como array.*

4.5.3 Preparação dos dados (function)

O node **function** `preparar_telemetria` (separador *On Message*) transforma o array de registos nos dados de cada gráfico:

- **Converte a data em milissegundos - s**ubstitui o espaço entre o dia e a hora por `T` e corta os microssegundos para três dígitos, para que o JavaScript a interprete corretamente. O resultado é o valor `x` de cada ponto.
- **Reduz o número de pontos - u**sa apenas um registo em cada 20 (`step = 20`), o que dá cerca de 900 pontos por gráfico em vez de 18250, mantendo o aspeto geral da curva.
- **Calcula a velocidade máxima** - percorrendo todos os registos (e não só os selecionados), o que dá 315 km/h.
- **Devolve cinco mensagens**, uma por saída - a velocidade, as rotações, o acelerador e a mudança (cada uma um array de pontos `{x, y}`, com um `topic` que dá nome à série) e a velocidade máxima (um único valor, para o indicador).

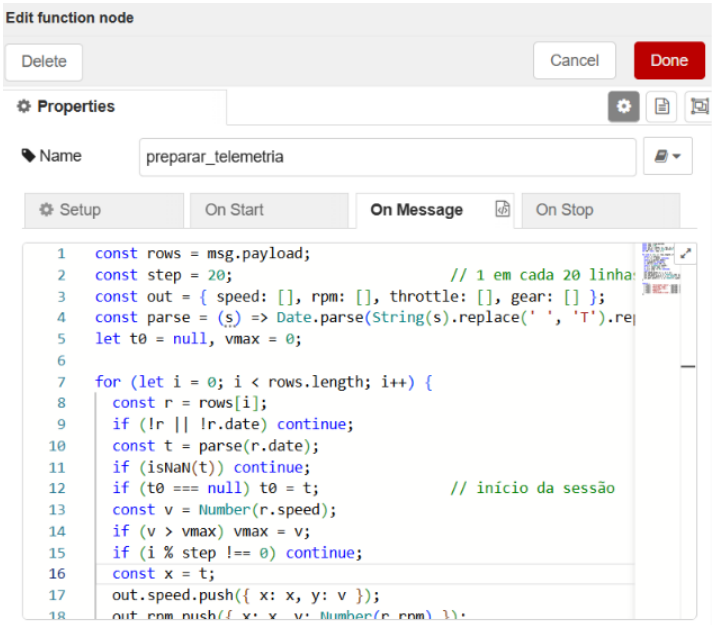


Figura 32 – *Node function preparar_telemetria (separador On Message): conversão das datas, redução de pontos e construção dos arrays.*

O código completo do node é o seguinte:

```
const rows = msg.payload;
const step = 20;                          // 1 em cada 20 linhas (~900 pontos)
const out = { speed: [], rpm: [], throttle: [], gear: [] };
const parse = (s) => Date.parse(String(s).replace(' ', 'T').replace(/(\.\d{3})\d*/, '$1'));
let t0 = null, vmax = 0;

for (let i = 0; i < rows.length; i++) {
  const r = rows[i];
  if (!r || !r.date) continue;
  const t = parse(r.date);
  if (isNaN(t)) continue;
  if (t0 === null) t0 = t;                // início da sessão
  const v = Number(r.speed);
  if (v > vmax) vmax = v;
  if (i % step !== 0) continue;
  const x = t;
  out.speed.push({ x: x, y: v });
  out.rpm.push({ x: x, y: Number(r.rpm) });
  out.throttle.push({ x: x, y: Number(r.throttle) });
  out.gear.push({ x: x, y: Number(r.n_gear) });
}

return [
  { topic: 'Velocidade (km/h)', payload: out.speed },
  { topic: 'Rotações (rpm)',    payload: out.rpm },
  { topic: 'Acelerador (%)',    payload: out.throttle },
  { topic: 'Mudança',           payload: out.gear },
  { topic: 'Velocidade máxima', payload: vmax }
];
```

4.5.4 Gráficos e indicador

Cada gráfico é um node **ui-chart** (tipo *Line*) ligado a uma das saídas da função. A configuração mais importante está no eixo e nas propriedades, que se vê na figura seguinte.

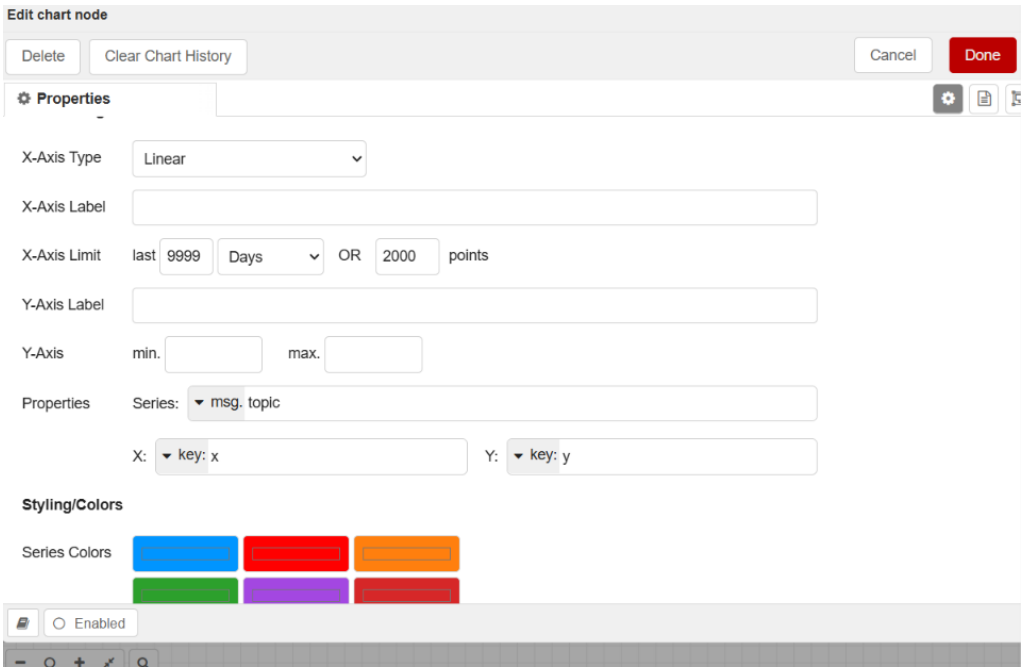


Figura 33 – *Configuração do eixo e das propriedades de um ui-chart: eixo X linear, limite de pontos e chaves x e y.*

A série é identificada por `msg.topic`, e os valores dos eixos são lidos das **chaves** `x` e `y` de cada objeto do array (*X: key x* e *Y: key y*). Por defeito, o node espera um número de cada vez e usa a hora de chegada da mensagem como X, o que não serve para desenhar de uma só vez uma série completa. O limite do eixo X está em 9999 dias ou 2000 pontos: os dados são de setembro de 2023 e o limite por defeito (a última hora) descartava todos os pontos, por serem muito antigos. A opção *Action* está em *Replace*, para que cada mensagem substitua os dados anteriores em vez de os acrescentar.

Nos gráficos da velocidade, das rotações e da mudança, o eixo X é do tipo *Linear*, pelo que mostra o instante de cada ponto em milissegundos desde 1 de janeiro de 1970 (por exemplo, 1 694 783 000 000, que corresponde às 13:03:20 UTC de 15 de setembro de 2023). No gráfico do acelerador, o eixo é do tipo *Timescale* e mostra diretamente a hora, o que é mais legível.

Os restantes gráficos (Rotações, Acelerador e Mudança) têm **a mesma configuração** que o da Velocidade, só mudam o nome, o título (*Label*), o grupo onde aparecem e, no caso do acelerador, o tipo de eixo X.

Saída da função e widget	Grupo	Título
1 – ui-chart Velocidade	Velocidade	Velocidade (km/h)
2 – ui-chart Rotações	Motor	Rotações (rpm)
3 – ui-chart Acelerador	Pedais	Acelerador (%)
4 – ui-chart Mudança	Motor	Mudança
5 – ui-gauge Velocidade máxima	Velocidade	Velocidade máxima

O indicador (**ui-gauge**) é do tipo *Half Gauge*, com agulha, intervalo de 0 a 350 km/h e zonas de cor (verde, amarelo e vermelho), e mostra a velocidade máxima da sessão: **315 km/h**. Os widgets estão organizados na página `Telemetria`, dividida nos grupos Velocidade, Motor e Pedais, o que permite apresentar os gráficos lado a lado.

4.5.5 Resultado

Depois de publicar o fluxo (*Deploy*), o dashboard fica disponível em `http://127.0.0.1:1880/dashboard`. As figuras seguintes mostram o resultado: o indicador da velocidade máxima e o gráfico da velocidade no grupo Velocidade, os gráficos das rotações e da mudança no grupo Motor, e o gráfico do acelerador no grupo Pedais.

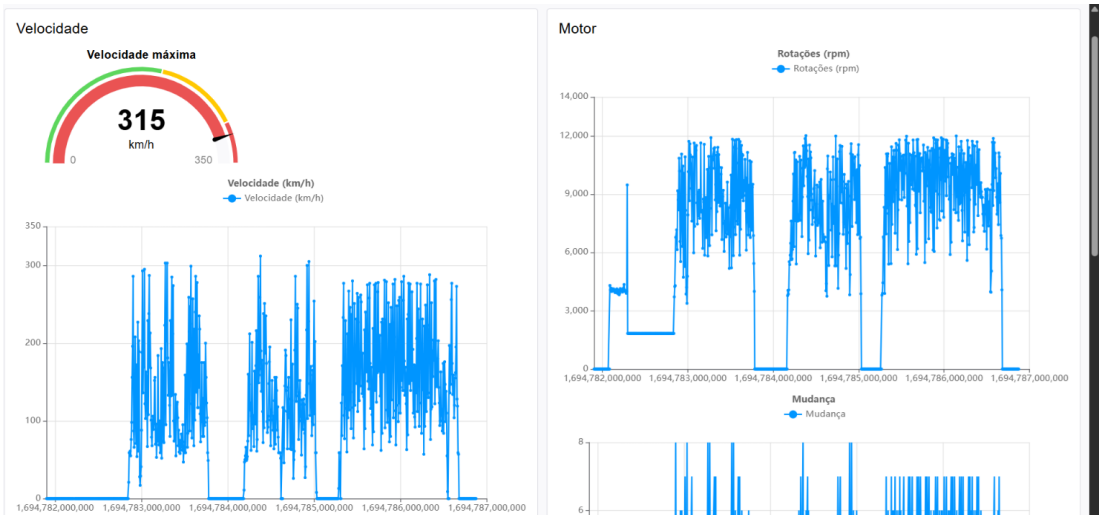


Figura 34 – *Dashboard: indicador da velocidade máxima e gráfico da velocidade (grupo Velocidade) e gráfico das rotações (grupo Motor).*

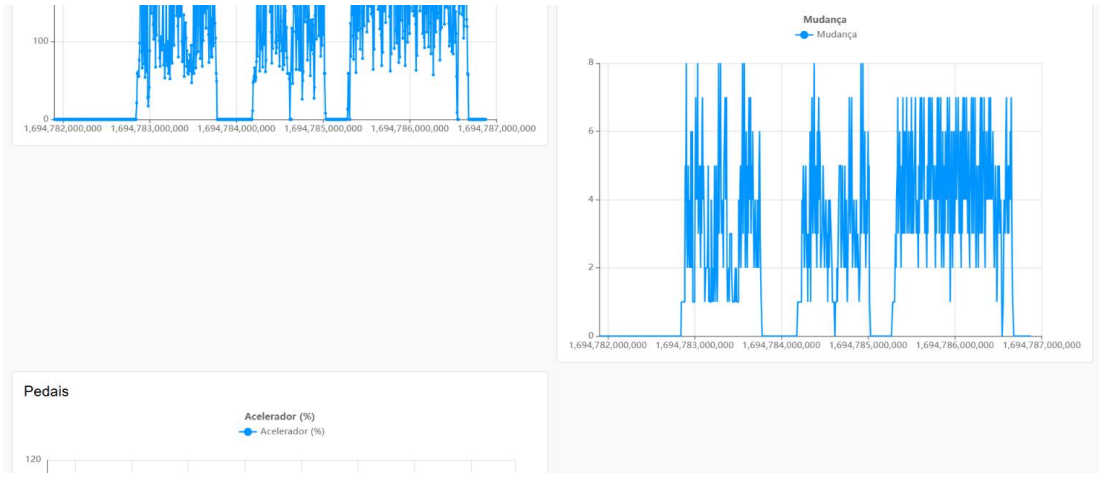


Figura 35 – *Dashboard (continuação): final do gráfico da velocidade, gráfico da mudança de caixa e início do grupo Pedais.*

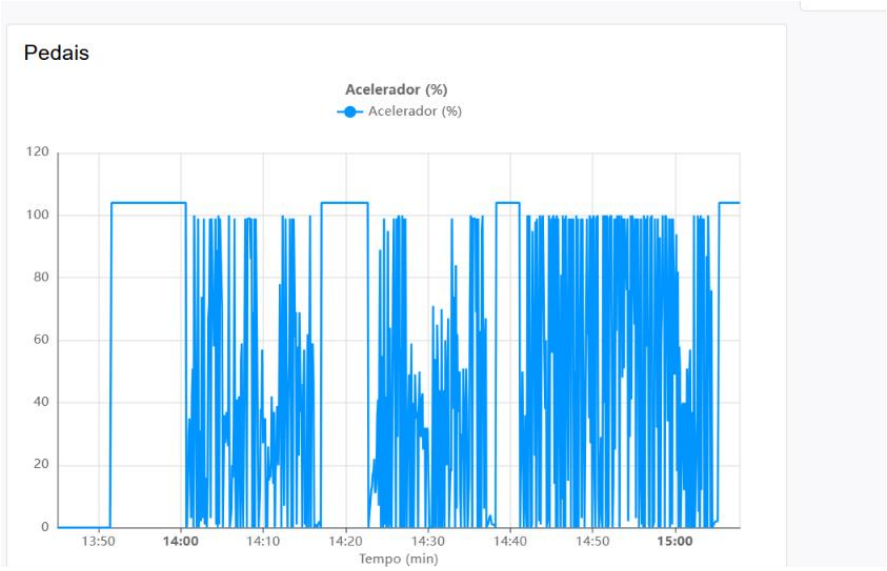


Figura 36 – *Dashboard: grupo Pedais, com o gráfico do acelerador ao longo da sessão (eixo de tempo em hora local).*

Os gráficos mostram **três períodos de atividade em pista**, separados por intervalos em que o carro está parado (velocidade, rotações e mudança a zero, possivelmente paragens na box). Durante a atividade, a velocidade chega perto dos 300 km/h, com um máximo de 315 km/h, as rotações sobem até cerca de 12 000 rpm, a mudança varia entre a 1.ª e a 8.ª, e o acelerador oscila entre 0 e 100 %.

Há ainda uma **observação sobre a qualidade dos dados**: nos intervalos em que o carro está parado, o gráfico do acelerador mostra um patamar de cerca de 104 %, acima do máximo de 100 %. Este valor existe nos dados de origem e a limpeza não o removeu, porque não foi definida nenhuma regra para este campo. Poderia ser tratado no Kettle com uma condição adicional (`throttle <= 100`) no passo de filtragem.

4.5.6 Problemas encontrados e soluções

Durante a construção do dashboard surgiram três problemas, resumidos na tabela seguinte.

Problema	Causa	Solução
O node file in dá o erro `ENOENT` (ficheiro não encontrado).	O caminho foi copiado do Explorador de Ficheiros com aspas, e o Node-RED trata-as como parte do nome do ficheiro.	Remover as aspas do início e do fim do caminho.
Gráficos e indicador sem dados: o debug mostra uma única coluna chamada `brake;date;driver_number;....`	O Text file output do Kettle grava com ponto e vírgula, e o nó csv esperava vírgulas.	Escolher o separador semicolon no nó csv.
Gráficos vazios, apesar de os dados chegarem à função.	Os valores por defeito do ui-chart (X = timestamp, Y = msg.payload) e o limite de uma hora não servem para uma série completa de dados de 2023.	Usar X e Y como chaves (`x` e `y`), Action em Replace e alargar o limite do eixo X.

5. Jobs

5.1 Job de envio do resumo por email

O job sequencia as transformaçóes: arranca com **Start**, corre a transformação de limpeza (**Transformation**), envia o resumo por email (**Mail**) e termina em **Success**. As setas verdes entre as entradas significam *Follow when result is true*: cada passo só corre se o anterior tiver terminado com sucesso, o que evita enviar um email com valores em branco quando a limpeza falha.



Figura 37 – *Diagrama do job: Start, Transformation, Mail e Success.*

5.2 Entrada Transformation

A entrada *Transformation* aponta para o ficheiro `.ktr` da transformação de limpeza e usa a configuração de execução *Pentaho local*, com a opção de esperar que a transformação termine antes de avançar. É por correr dentro deste job que as variáveis do passo `guardar_variaveis` ficam disponíveis para o email.

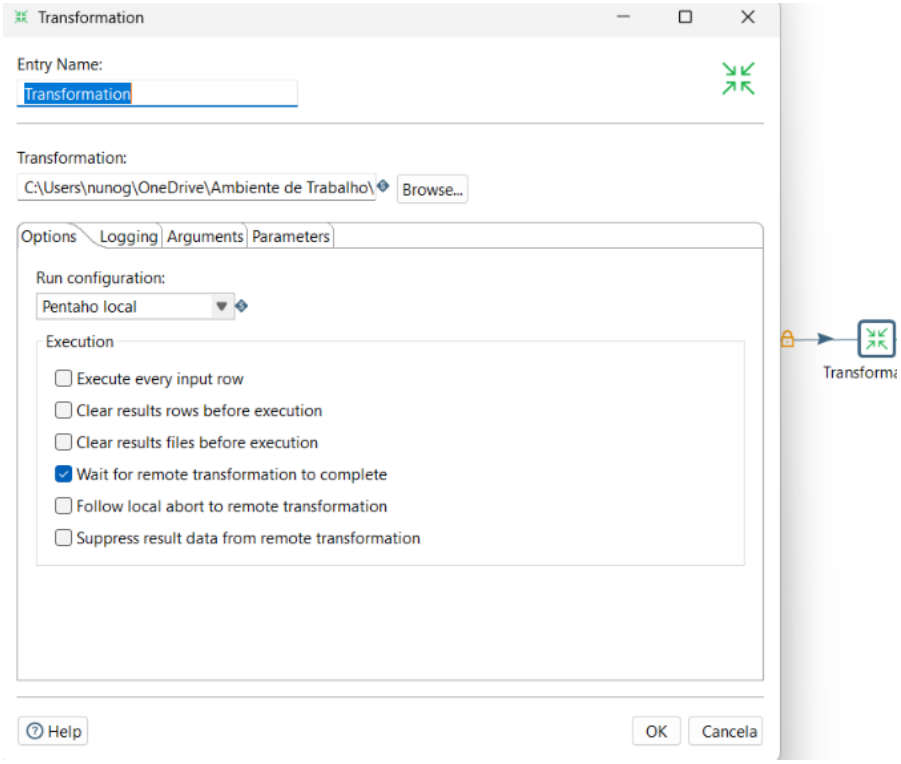


Figura 38 – *Configuração da entrada Transformation do job.*

5.3 Entrada Mail

A entrada *Mail* está configurada em três separadores. No separador **Addresses** definem-se o destinatário e o remetente. No separador **Server** indicam-se o servidor SMTP do Microsoft 365 (`smtp.office365.com`), a porta 587, a autenticação com utilizador e palavra-passe e a ligação segura do tipo TLS. No separador **EMail Message** escrevem-se o assunto e o corpo da mensagem, que usa as variáveis calculadas na transformação, na forma `${W_TOTAL}`, `${W_OK}` e `${W_PCT_OK}`, entre outras. O Kettle substitui-as pelos valores no momento do envio.

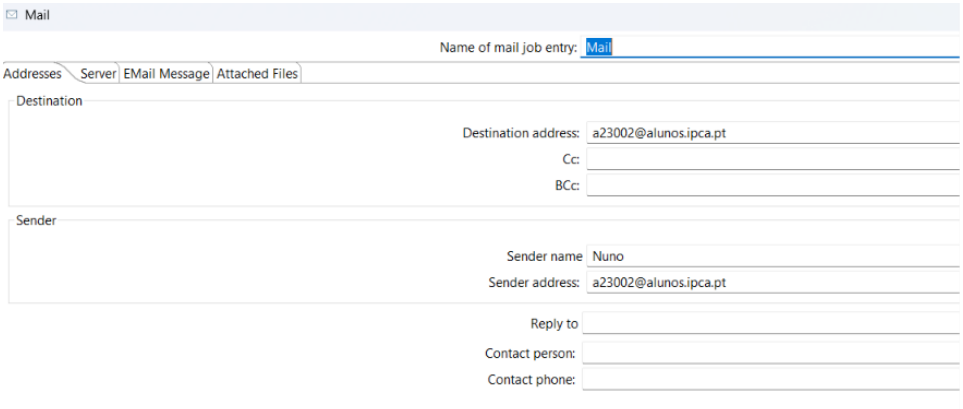


Figura 39 – *Separador Addresses: destinatário e remetente.*

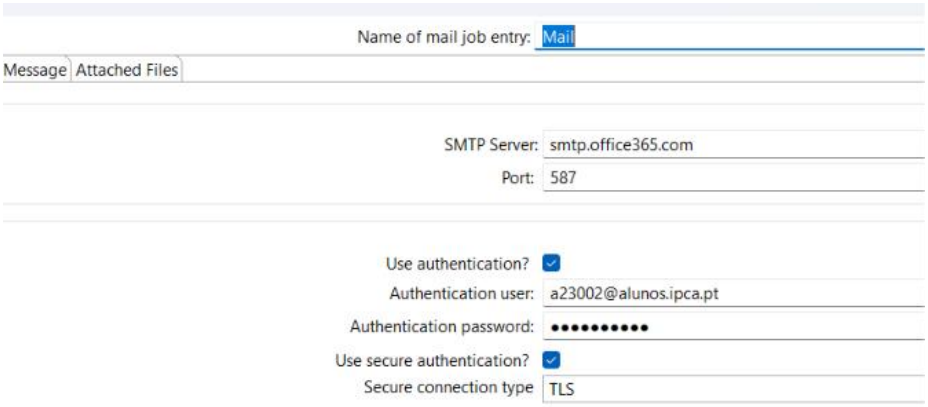


Figura 40 – *Separador Server: smtp.office365.com, porta 587, autenticação e TLS.*

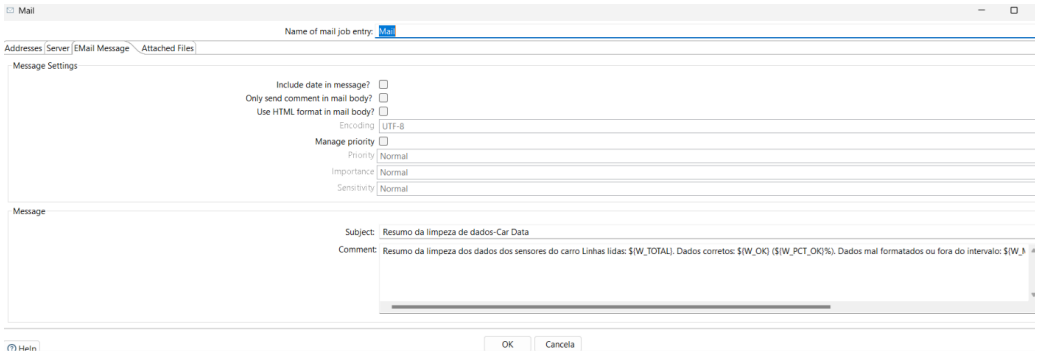


Figura 41 – *Separador EMail Message: assunto e corpo com variáveis (${W_TOTAL}, ${W_OK}, ${W_PCT_OK}, ...).*

5.4 Resultado

A execução do job envia o email com os valores já calculados. No exemplo, referente ao ficheiro car data, o resumo indica 18620 linhas lidas, das quais 18250 corretas (98%), 220 mal formatadas ou fora do intervalo (1,2%) e 150 duplicadas (0.8%), valores que coincidem com os erros introduzidos. O Kettle acrescenta automaticamente ao fim da mensagem a identificação do job e da entrada que enviou o email.

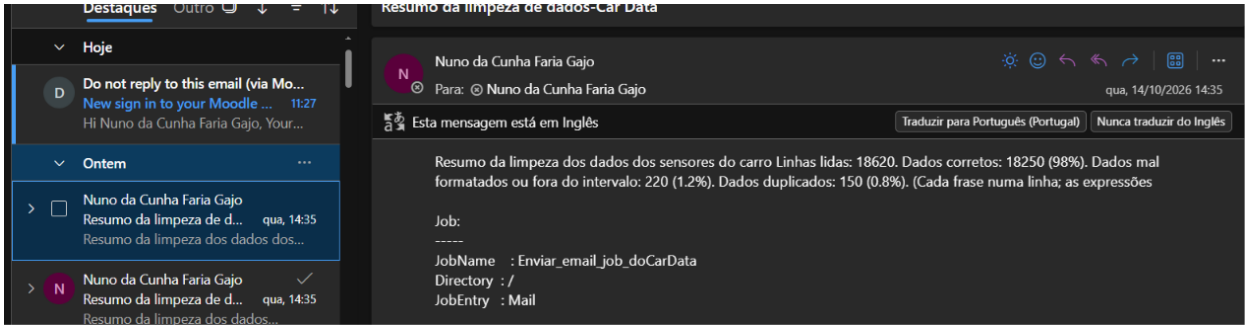
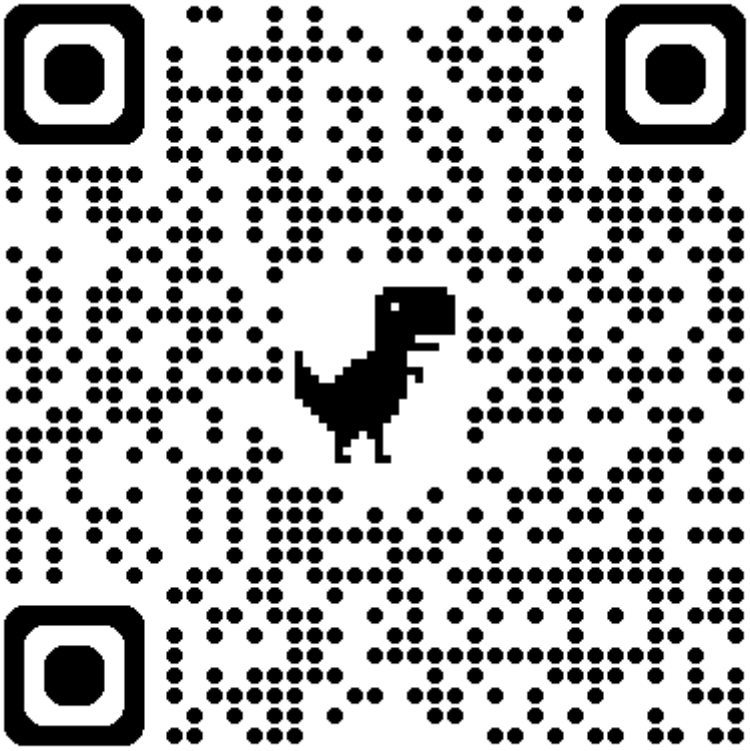


Figura 42 – *Email recebido com o resumo da limpeza dos dados do carro.*

6. Demonstração

Vídeo com demonstração (QR Code)



7. Conclusão e trabalhos futuros

7.1 Conclusão

O trabalho demonstrou que o Kettle permite construir, com passos simples e sem escrever código para a limpeza em si, um processo completo de tratamento de dados telemétricos. Foram tratados dois formatos de origem (CSV e JSON), obtidos dados diretamente de uma API e produzido automaticamente um resumo do resultado da limpeza, enviado por email. Em todos os casos o número de linhas rejeitadas em cada fase coincidiu com o número de erros introduzidos: no CSV, 370 linhas removidas (2,0 %) e 98,0 % de dados corretos. Os dados limpos foram ainda apresentados num dashboard em Node-RED, com gráficos da velocidade, das rotações, do acelerador e da mudança de caixa, e um indicador da velocidade máxima.

Entre as lições retiradas destacam-se:

- Leitura de campos como texto e converter num passo posterior, para que os erros não interrompam a execução
- Encaminhar os rejeitados para passos próprios em vez de os descartar, o que permite contá-los
- Indicar explicitamente o separador decimal para evitar leituras erradas dependentes da língua do sistema
- Correr as transformações dentro de um job para que as variáveis cheguem ao email
- Configurar o Node-RED de acordo com os dados: o separador do CSV (ponto e vírgula) e o mapeamento dos eixos dos gráficos (chaves x e y).

7.2 Trabalhos futuros

- **Exportar os dados limpos para uma base de dados** e importar dados de uma base de dados para o Kettle.
- **Receber dados em streaming** (Node-RED e broker MQTT, como o Mosquitto) e integrá-los no Kettle.
- **Contingência na obtenção de dados:** tentar primeiro a API e, só se falhar, usar o ficheiro local, através de caminhos de sucesso e de falha no job.
- **Estender o dashboard** com a comparação entre pilotos (por exemplo, o 55 com o 44), com os dados meteorológicos e com uma regra de limpeza para valores do acelerador acima de 100 %.

8. Referências bibliográficas

OpenF1. (s.d.). *OpenF1 – API de dados de Fórmula 1*. https://openf1.org

OpenF1. (s.d.). Endpoint `car_data` (telemetria). https://api.openf1.org/v1/car_data?driver_number=55&session_key=9159

OpenF1. (s.d.). Endpoint `weather` (meteorologia). https://api.openf1.org/v1/weather?meeting_key=1219

Hitachi Vantara. (s.d.). *Pentaho Data Integration – documentação*. https://docs.hitachivantara.com

Microsoft. (s.d.). *How to set up a multifunction device or application to send email using Microsoft 365 or Office 365*. https://learn.microsoft.com/exchange/mail-flow-best-practices/how-to-set-up-a-multifunction-device-or-application-to-send-email-using-microsoft-365-or-office-365

Oracle. (s.d.). *Java Platform, SE – java.util.regex.Pattern*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/regex/Pattern.html

Goessner, S. (2007). *JSONPath – XPath for JSON*. https://goessner.net/articles/JsonPath/

Node-RED. (s.d.). *Node-RED – documentação*. https://nodered.org/docs/

FlowFuse. (s.d.). *Node-RED Dashboard 2.0 – documentação*. https://dashboard.flowfuse.com